Présentation de projet :
Réseau Télémétrique de Mesures en Temps Réel.
Mesures Sismiques
R.T.M.T.R_1

Etudiant 1

Présentation du projet

- Création d'un réseau de stations sismiques pour fournir des données en temps réel sur les activités sismiques d’une zone géographique
- Stations connectées pour former un observatoire en ligne des activités sismiques
- Intégration au réseau existant pour couvrir une zone plus étendue
- Mettre les données à disposition des scientifiques du domaine.

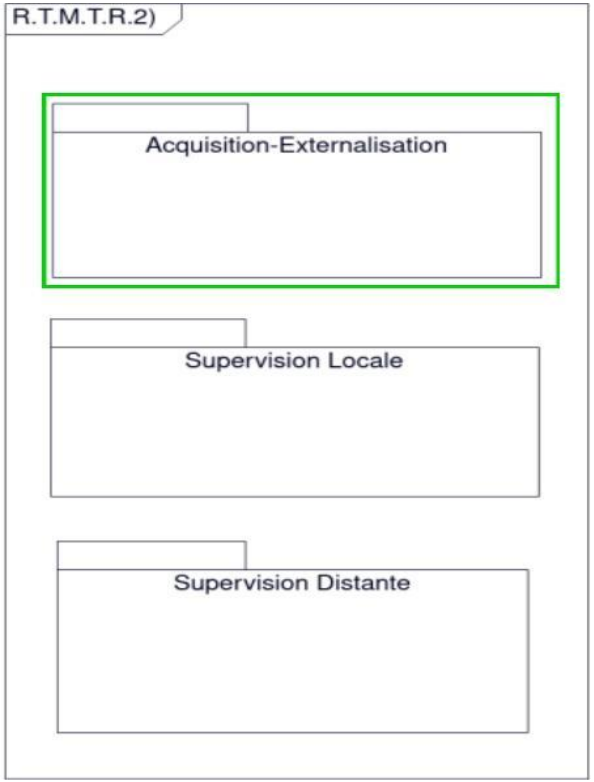
Missions de l’application à réaliser

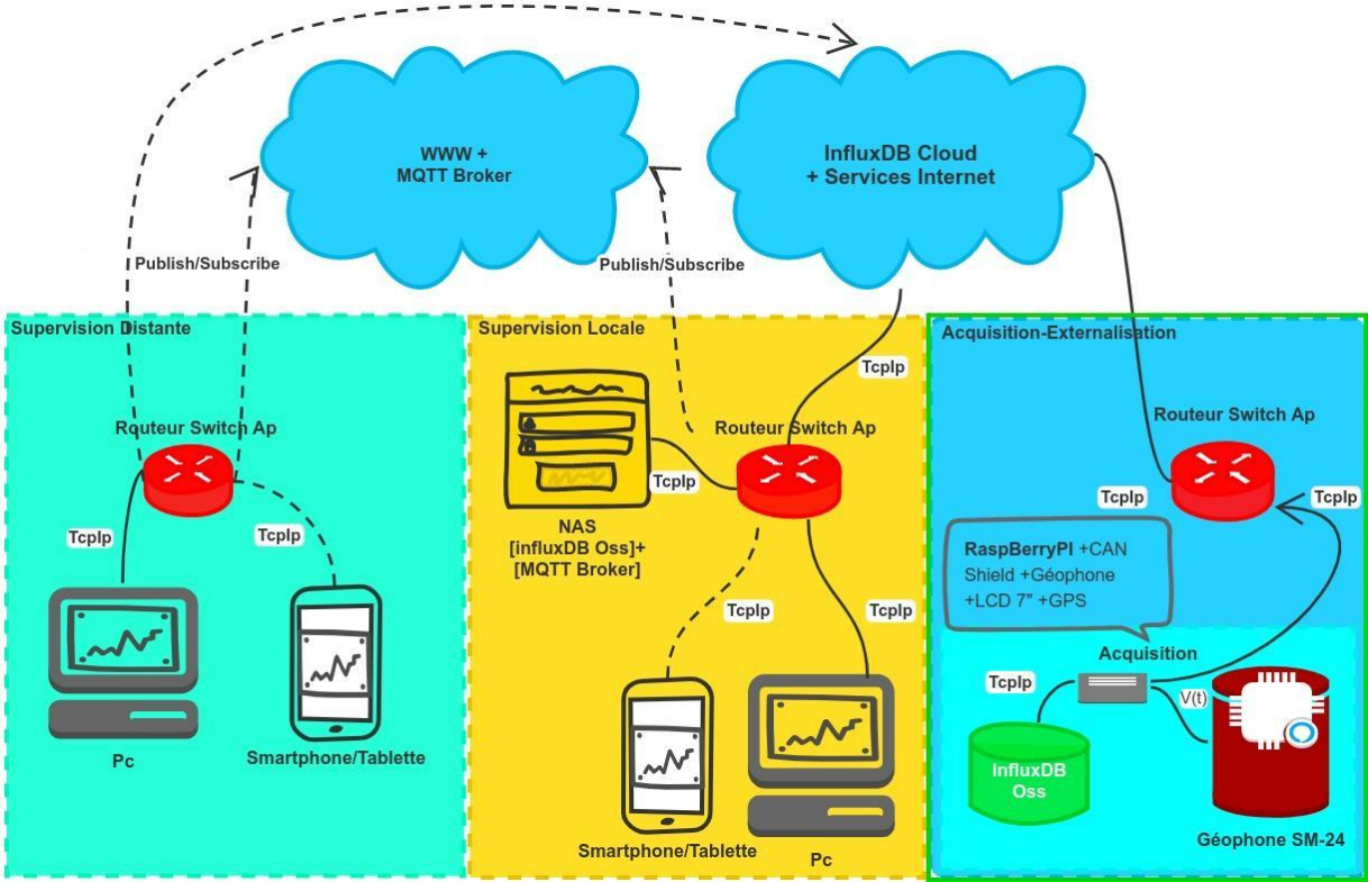
- Transférer périodiquement nos relevés de données et de position à un serveur.
- Présenter ces données sous forme de graphes.
- Enregistrer ces périodes d’observation afin de les transmettre ultérieurement au serveur en cas de coupure.
- Observation en temps réel de l'activité sismique et consultation des données historiques grâce à la base de données intégrée dans chaque station sismique.

Spécification

- Nous allons découper notre système en 3 parties distinctes :
  - Acquisition et Externalisation des données
  - Supervision Locale
  - Supervision Distante

- De mon côté, je travaillerai sur la partie Acquisition et Externalisation des données.





Synoptique RTMTR

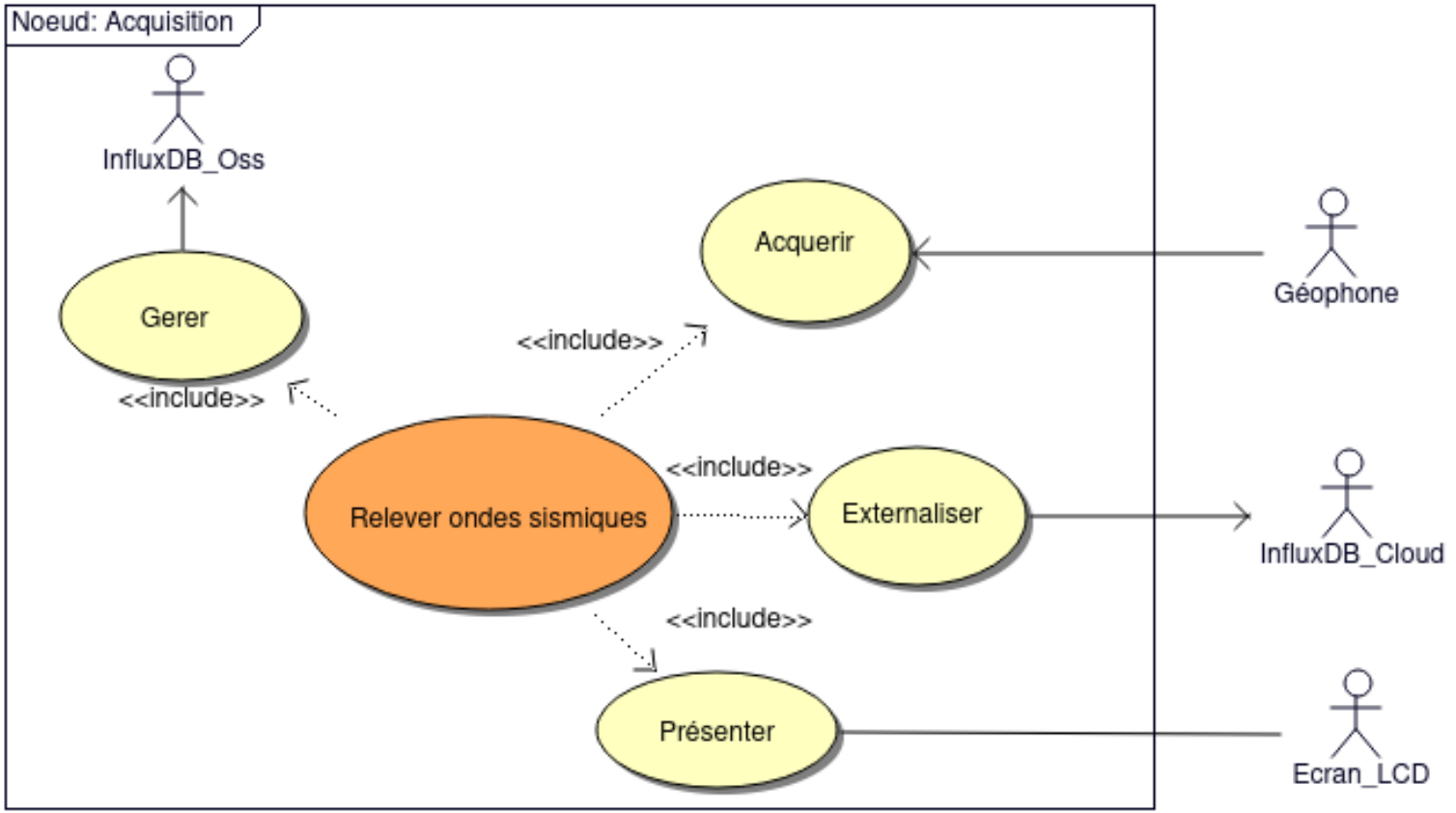


Diagramme des cas d’utilisation

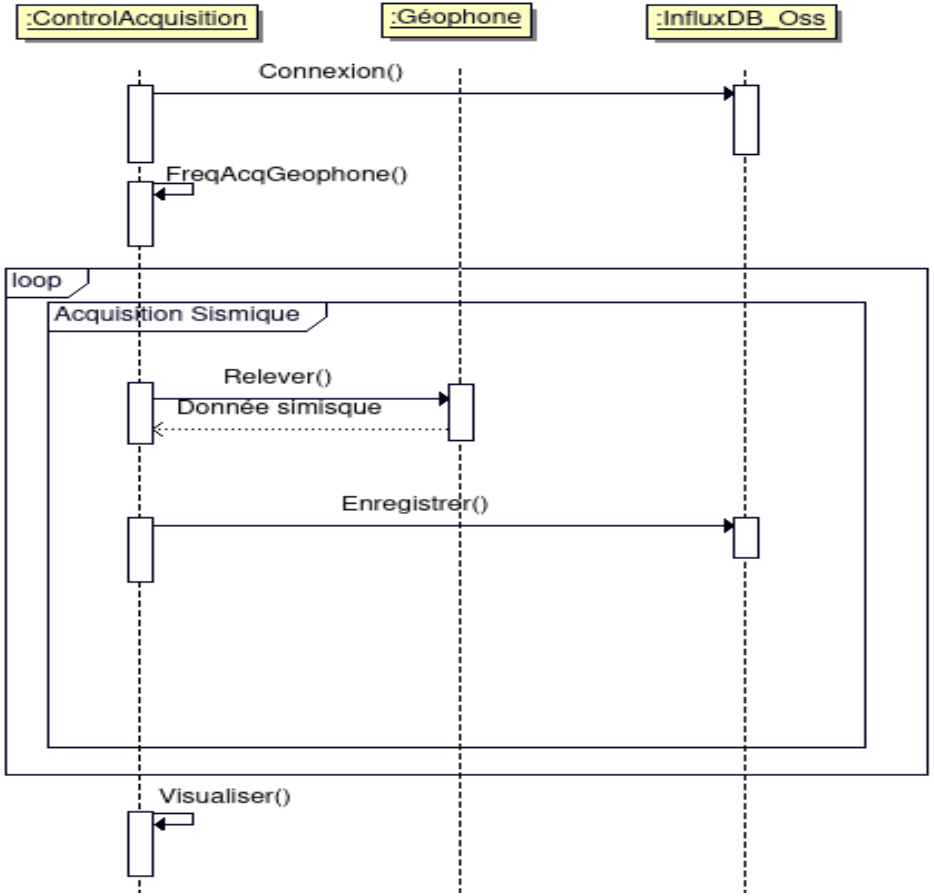


Diagramme de séquence

Description et rôle des acteurs

- InfluxDB_OSS
  - Permet l'enregistrement des mesures sismiques localement. Serveur InfluxDb_Oss installé sur le RaspberryPI.

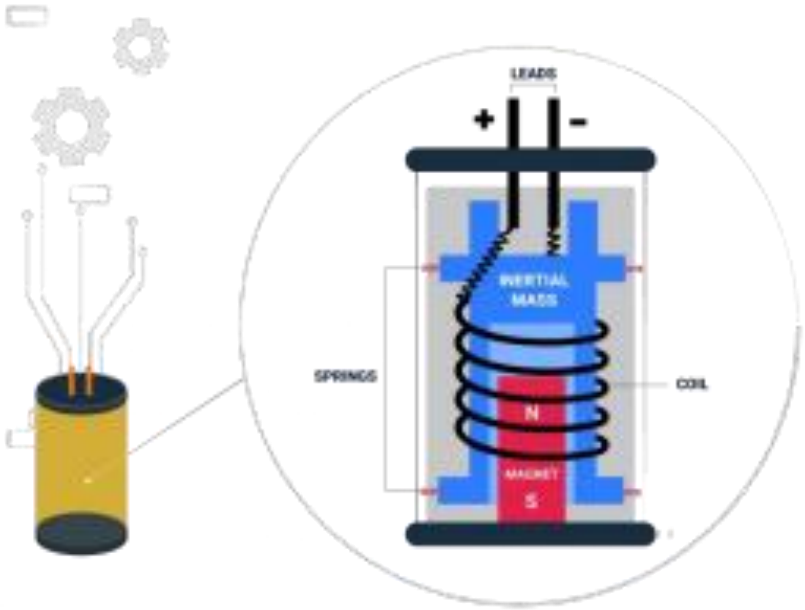
- Géophone
  - Capteur d'ondes sismiques

- Ecran_LCD
  - Permet de tester le bon fonctionnement de la station, et de régler la fréquence d'arrivée des données.

- InfluxDB_Cloud
  - Permet l'externalisation des mesures sismiques dans un cloud gérer par InfluxDB_Cloud

Fonctionnement du capteur sismique : Le géophone

- Appareil qui mesure les vibrations de la Terre (tremblements de terre, explosions).
- Génère une tension proportionnelle au mouvement en utilisant un aimant à ressort qui induit un champ magnétique dans une bobine.
- Utilisé principalement pour l'exploitation minière et l'exploration.



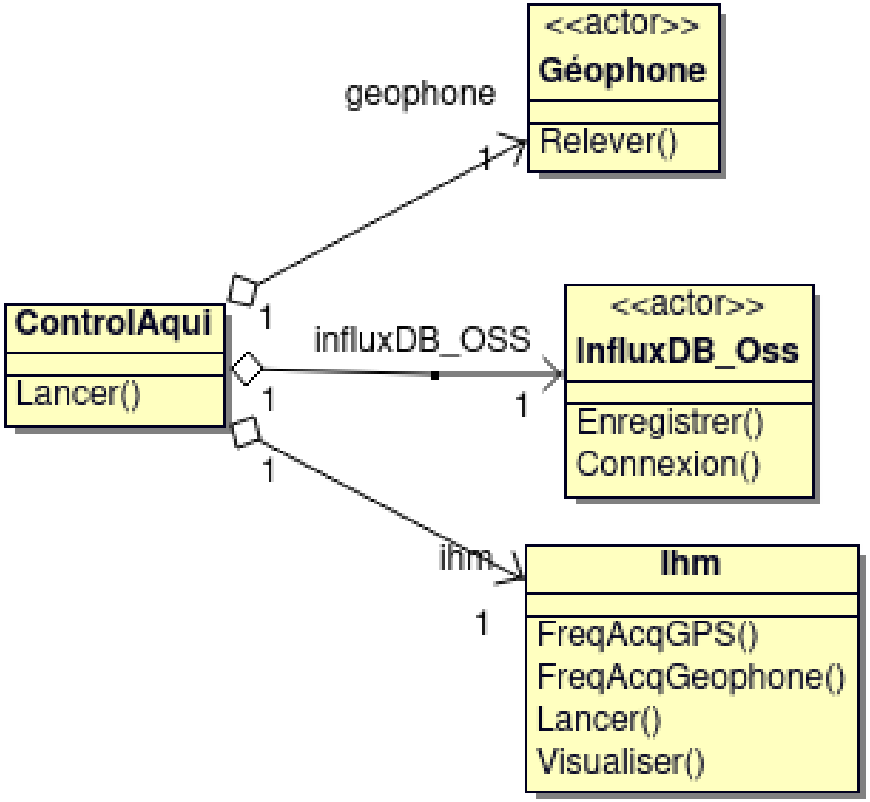
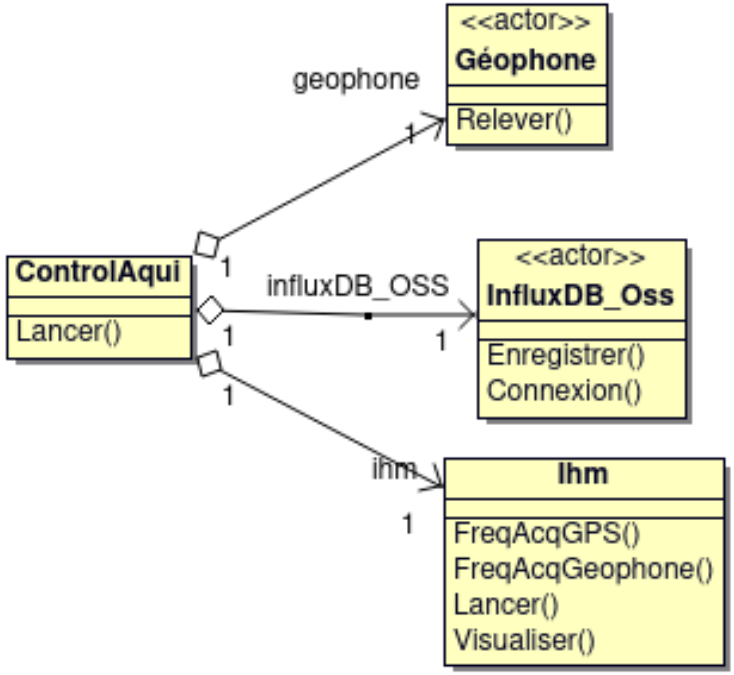


Diagramme de classes

Conception de la classe Geophone

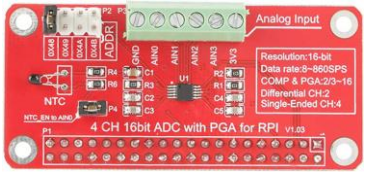


Prise en compte des contraintes matérielles et logicielles

Contraintes matérielles :



- Un Raspberry Pi 4.
- Convertisseur Analogique Numérique (CAN) Shield ADS1115 pour Raspberry Pi.

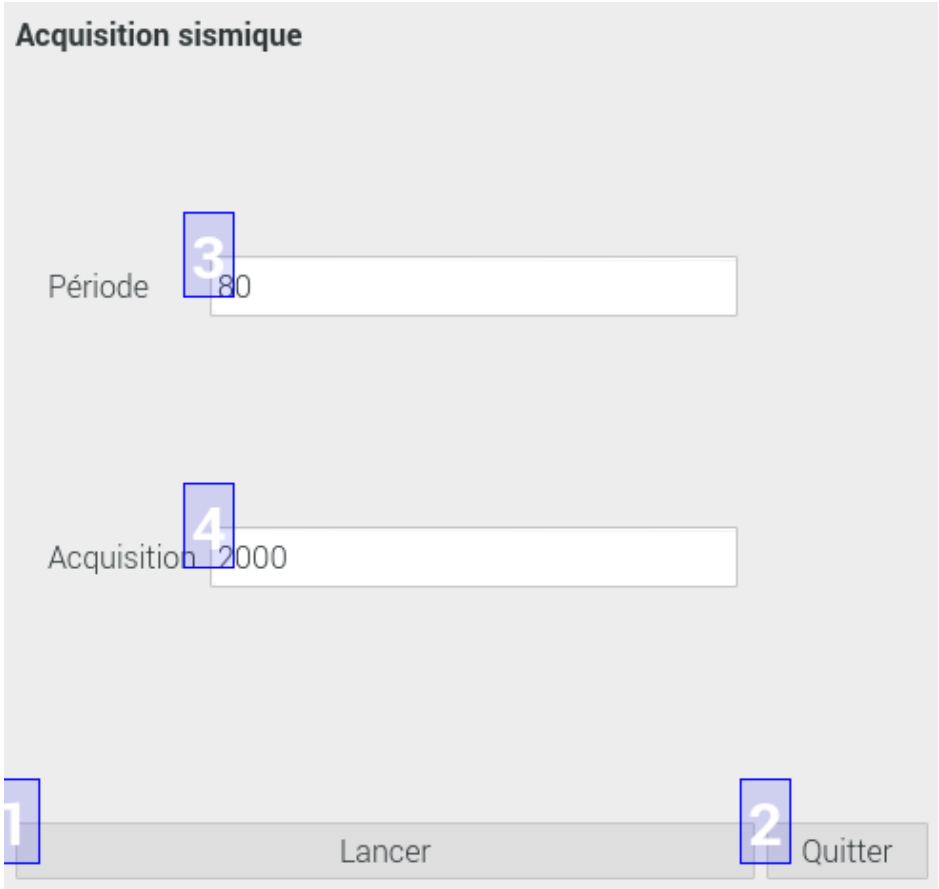


- Géophone SM-24



Contraintes logicielles :

- Classes clientes InfluxDB
- Classe CAN_ADS1115
- C++ Qt5



1. Bouton d'exécution

1. Fermeture de l'application

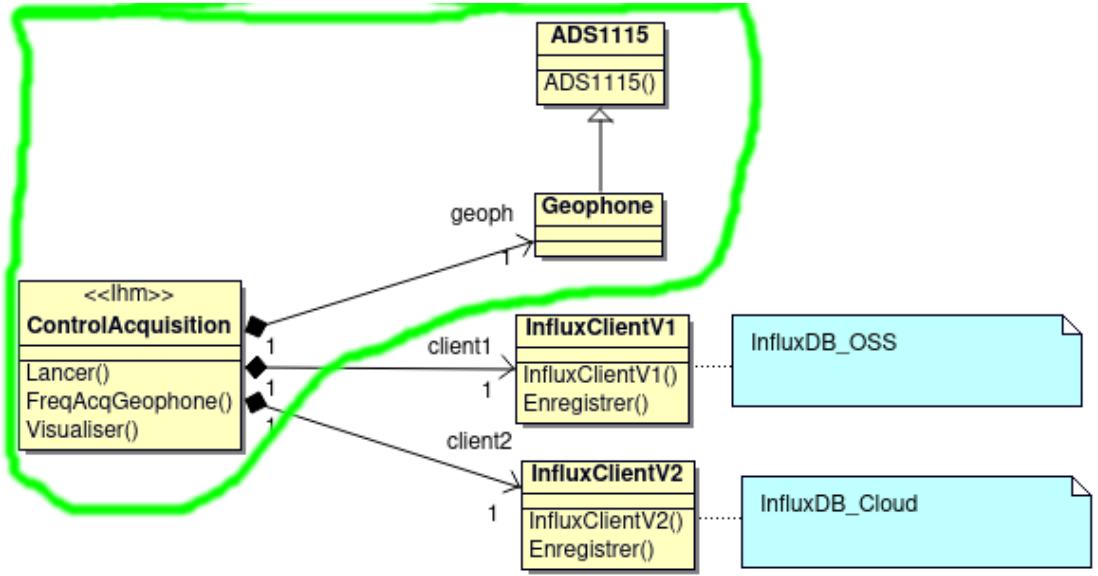
1. Ligne d’édition période d’acquisition pour les ondes sismiques

1. Ligne d’édition de la valeur acquise (forme numérique)

Prototype de l’IHM test pour valider la classe Geophone

Diagramme de classe de conception pour le géophone

- Le géophone SM-24 produit une tension qui doit être relevée à l'aide du CAN ADS1115. C'est pourquoi j'utilise l'héritage pour disposer de l'ensemble des fonctions de la classe ADS1115 me permettant d'observer les variations sismiques générées par le géophone



Extrait de code de la classe Geophone (.h)

```
#include <ADS1115.h>

class Geophone : public ADS1115
{
public:
    Geophone(uint8_t i2cAddress = ADS1015_ADDRESS);
};
```

Héritage de la classe ADS1115 fournie

Extrait de code de la classe Geophone (.cpp)

```
Geophone::Geophone(uint8_t i2cAddress):ADS1115 (i2cAddress)
{

}
```

Extrait de code de l'IHM test

```
void Widget::Relever()
{

    qDebug()<<"Relever";
      adc0 = geoph.readADC_Differential_0_1();
      ui->lacqui->setText(QString::number(adc0));
```

1. Le géophone fournit des variations de tensions positives et négatives.

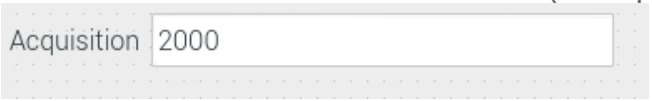
1. Il faut configurer le CAN pour une acquisition en mode différentiel.

adc0 = **geoph.readADC_Differential_0_1();**

1. L'acquisition devient possible par l'appel de la méthode

ui->lacqui->**setText(QString::number(adc0));**

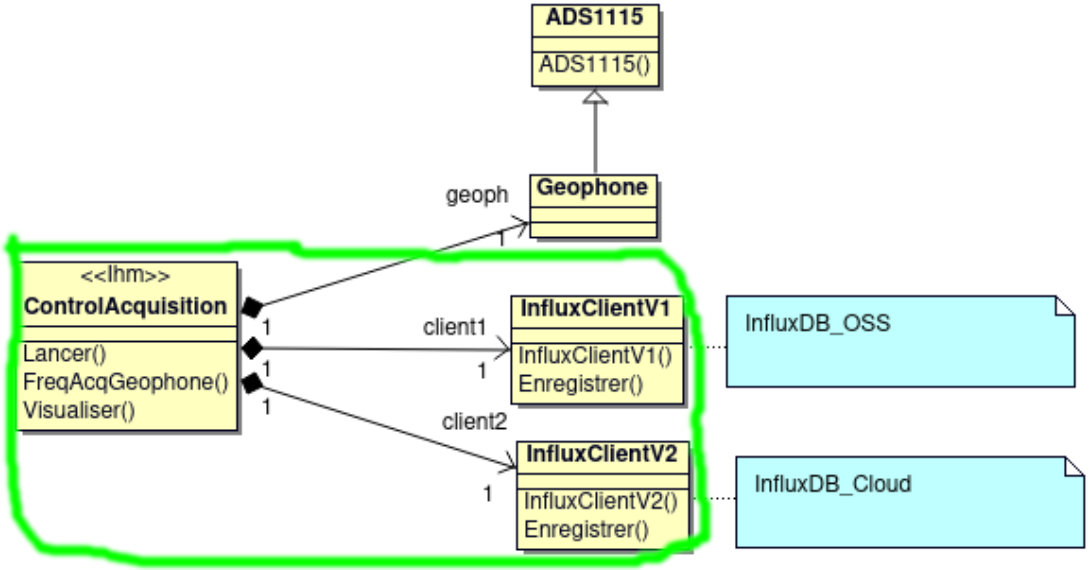
1. Pour obtenir sur l'IHM le résultat suivant (exemple) :



1. Conclusion, la classe **Geophone** a été validée

Diagramme de classe de conception pour l'accès aux bases de données

- Deux classes InfluxDB sont nécessaires car le Raspberry embarque un serveur InfluxDB V1 et l'autre classe est nécessaire car le NAS distant embarque un serveur InfluxDB V2.



Structuration des bases données locales et distantes

Deux classes InfluxDB sont nécessaires car le Raspberry embarque un serveur InfluxDB V1 et l'autre classe est nécessaire car le NAS distant embarque un serveur InfluxDB V2.

L'équipe a décidé de structurer les bases de données de la façon suivante :

- Structure de la base de donnée embarquée

  bd -> measurement -> field

  InfluxDBV1 définit trois zones

  1. bd : choix du nom de la base de données
  2. measurement : choix du nom du measurement
  3. field : choix des noms des mesures effectives

- Conception de la base de données embarquée

  bd -> measurement -> field

  iotsismique -> idstation1 -> mesure
  latitude
  longitude

  InfluxDBV1 affectation des trois zones

  1. iotsismique : nom de la base
  2. idstation : nom du measurement
  3. a) nom du field : mesure
     b) nom du field : latitude
     c) nom du field : longitude

Pour la base de données InfluxDBV2, le procédé est le même à part qu'il faut simplement remplacer "bd" par "bucket"

Extrait de code des classes InfluxDBV1 et InfluxDBV2 (.h)

Implémentation pour la V1

```
#include <QObject>
#include <influxdb.hpp>


class InfluxClientV1 : public QObject
{
    Q_OBJECT
public:
    explicit InfluxClientV1(QObject *parent = nullptr);
    void Enregistrer(int data);
};
```

Implémentation pour la V2

```
#include <influxdbV2.hpp>
```

Extrait de code des classes InfluxDBV1 et InfluxDBV2 (.cpp)

```cpp
void InfluxClientV1::Enregistrer(int data)
{

    influxdb_v1::server_info si("127.0.0.1", 8086, "iotsismique", "
    influxdb_v1::builder()
    .meas("idstation1")
    .field("mesure", data)
    .field("latitude", 14)
    .field("longitude", -61)
    //.timestamp(1512722735522840439)
    .post_http(si);

}
```

identifiants masques

```cpp
void InfluxClientV2::Enregistrer(int data)
{

    influxdb_v2::server_info si("192.168.1.200", 49153, "btssnir"
                                                        , "iotsismique");
    influxdb_v2::builder()
    .meas("idstation1")
    .field("mesure", data)
    .field("latitude", 14)
    .field("longitude", -61)
    //.timestamp(1512722735522840439)
    .post_http(si);

}
```

jeton masque

Les méthodes Enregistrer() des deux classes sont similaires.
Seuls les connecteurs “server_info si(...,...,...);” sont différents.

Utilisation de la classe Geophone pour valider l'enregistrement dans les deux bases de données

Évolution de l'IHM test pour l'acquisition puis l'enregistrement.

Extrait de code de widget.h (IHM)

```
#include <QWidget>
#include <QTimer>
#include <geophone.h>
#include <influxclientv1.h>
#include <influxclientv2.h>

class Widget : public QWidget
{
    Q_OBJECT
public:
    Widget(QWidget *parent = nullptr);
    ~Widget();
    void setDark();
private slots:
    void on_blancer_clicked();
private:
    Ui::Widget *ui;
    Geophone geoph;
    QTimer *timer;
   int adc0;
   InfluxClientV1 client1;
   InfluxClientV2 client2;
public slots:
    void Relever();
};
```

Extrait de code de widget.cpp (IHM)

```
void Widget::Relever()
{

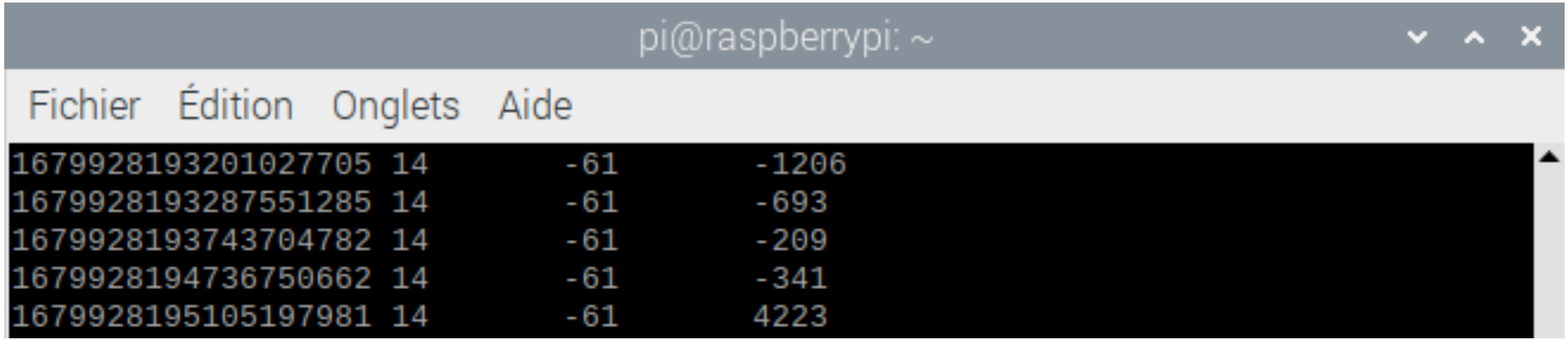
    qDebug()<<"Relever";
      adc0 = geoph.readADC_Differential_0_1();
      ui->lacqui->setText(QString::number(adc0));
      client1.Enregistrer(adc0);
      client2.Enregistrer(adc0);

}
```

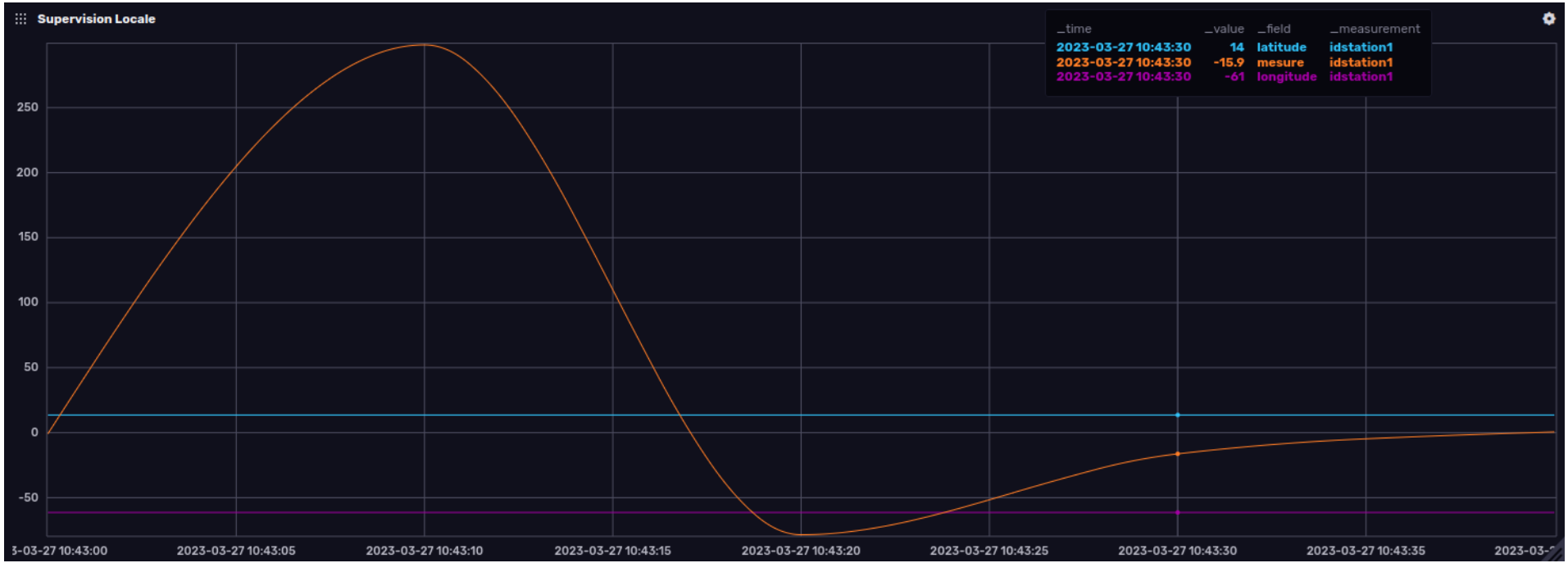
Acquisition d'une donnée sismique (un échantillon)

Enregistrement de la donnée acquise dans les bases de données (locale et distante)

Validation de la classe InfluxDBV1



Validation de la classe InfluxDBV2



Conclusion, les classes InfluxDBV1 et InfluxDBV2 ont été validées

Utilisation de QSimpleTickerGraph

```
ui->plot->setRange(-10000,10000);
ui->plot->setGridPitch(400);
```

Choix de l'échelle de la grille

```
void Widget::on_bzoomin_clicked()
{
    ui->plot->setPointWidth(++widthPoint);
}

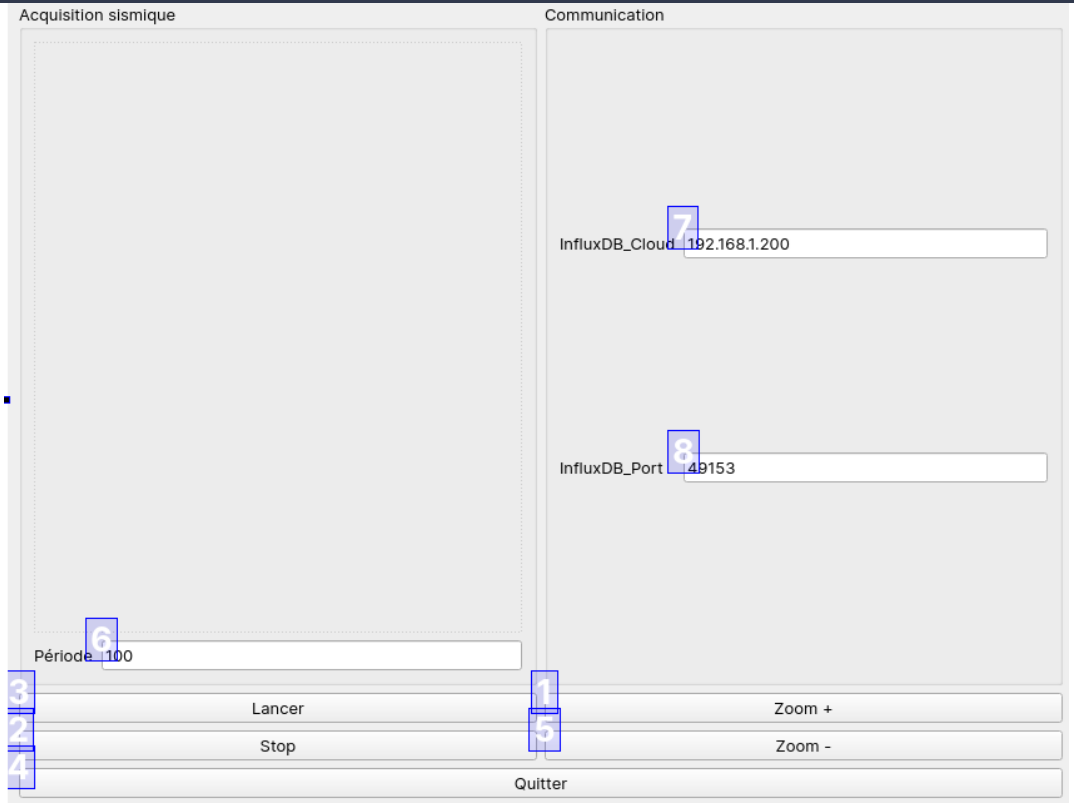
void Widget::on_bzoomout_clicked()
{
    ui->plot->setPointWidth(--widthPoint);

}
```

Codage du bouton de Zoom avant

Codage du bouton de Zoom arrière

Configuration de l’UI finale



1) Zoom avant

2) Arrêt du programme

3) Exécution de l’application

4) Fermeture de l'application

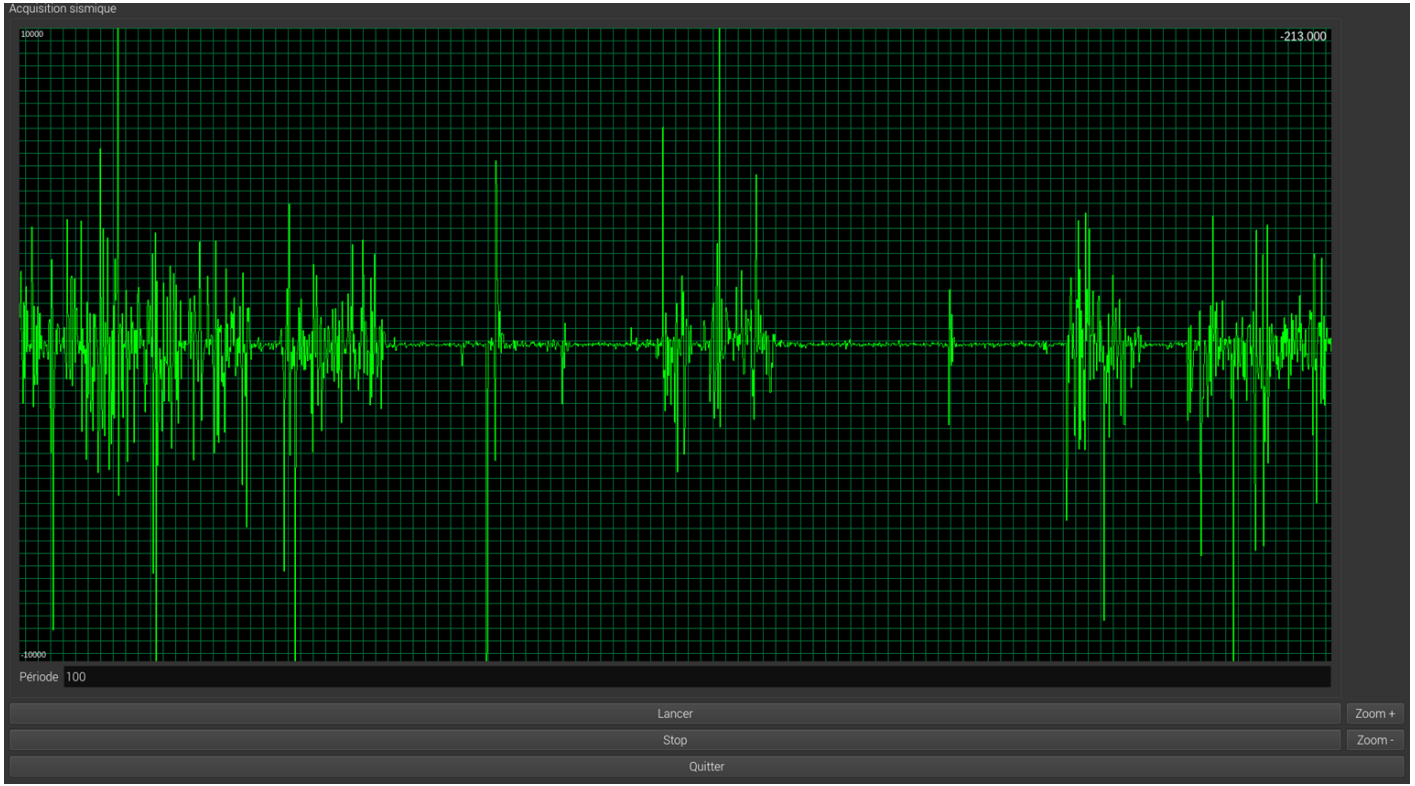
5) Zoom arrière

6) Édition de la période d'échantillonnage

7) Adresse IP du serveur distant

8) Port du serveur distant

Validation de QSimpleTickerGraph



Conclusion

Le projet a progressé avec succès jusqu'à présent, toutes les classes ont été validées et jugées fonctionnelles, établissant ainsi les fondations de l'application et du système.

- La division initiale entre le traitement des données de mouvement sismique et celui du positionnement GPS de la station sera unifiée dans la livraison finale.

- Ma démonstration finale se concentrera sur la validation de ma partie, tandis que mon collègue sera chargé de présenter et de valider la fonctionnalité d'acquisition de position GPS.